JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY

SCHOOL OF OPEN, DISTANCE AND eLEARNING
P.O. Box 62000, 00200
Nairobi, Kenya
E-mail: elearning@jkuat.ac.ke

ICS 2302 SOFTWARE ENGINEERING 1

This presentation is intended to covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green Exercise tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elearning@jkuat.ac.ke.

LESSON 6

Software Requirements

Learning Outcomes

A study of this lesson should enable students to:

- Describe the software requirements

6.1. Introduction

The problems that software engineers have to solve are often immensely complex. Understanding the nature of the problems can be very difficult, especially if the system is new. Consequently, it is difficult to establish exactly what the system should do. The descriptions of the services and constraints are the requirements for the system and the process of finding out, analyzing, documenting and checking these services and constraints is called *requirements engineering.*

The term requirement is not used throughout the software industry in a consistent way. In some cases, a requirement is seen as a high level, abstract statement of a service that the system should provide here referred to as user requirement. At other extreme, it is detailed, mathematically formal definition

of the system function here referred to as system requirement. Hence we can make the following definitions:

User requirements are statements, in natural language plus diagrams of what services the system is expected to provide and the constraints under which it must operate. It is written for the client and contractor mangers who do not have a detailed technical knowledge of the system.

System requirements set out the system services and constraints in detail. It is written for the senior technical and project managers.

Software design specification is an abstract description of the software design which is a basis for more detailed design and implementation. It is written for the software engineers who will develop the system.

6.2. Functional and Non Functional requirements

Software system requirements are often classified as functional or non functional requirements or as domain requirements.

Functional requirements

The functional requirements for a system describe the functionality or services that the system is expected to provide. These depend on the type of software which is being developed, the expected users of the software and the type of system which is being developed.

Functional requirements for a software system may be expressed in a number of different ways. Here are a number of functional requirements for a university library system for students and faculty to order books and documents from other li-

◀◀ ▶▶
◀ ▶
◀ Doc Doc ▶
Back Close

braries:

1. The user shall be able to search either all of the initial set of databases or select a subset from it.
2. The system shall provide appropriate viewers for the user to read documents in the document store.
3. Every order shall be allocated a unique identifier (ORDER ID) which the user shall be able to copy to the account's permanent storage area.

These are functional user requirements which define specific facilities which must be provided by the system. In principle the functional requirements specification of a system should be both complete and consistent. Completeness means that all services required by the user should be defined. Consistency means that requirements should not have contradictory definitions.

◀◀ ▶▶ ◀ ▶ ◀ Doc Doc ▶ Back Close

Non-functional requirements

Non functional requirements, as the name suggests are those requirements which are not directly concerned with the specific functions delivered by the system. They may relate to emergent system properties such as reliability, response time and store occupancy.

Many non functional requirements relate to the system as a whole rather than to individual system feature. This means that they are often more critical that individual functional requirements. While a failure to meet an individual functional requirement may degrade the system, failure to meet a non-functional system requirement may make the whole system unusable .e.g. if an aircraft system does not meet its reliability requirements, it will not be certified as safe for operation.

Non functional requirements arise through user needs, because of budget constraints, organizational policies, need for interoperability regulations, privacy legislation etc.

Domain requirements

Domain requirements are derived from the application domain of the system rather than from the specific needs of system users. They may be new functional requirements in their own right, constrain existing functional requirements or set out how particular computations must be carried out. Domain requirements are important because they often reflect fundamental of the application domain. If these requirements are not satisfied, it may be impossible to make the system work satisfactorily.

User Requirements

The user requirements for a system should describe the functional and non-functional requirements so that they are understandable by system users who don't have detailed technical knowledge. They should only specify the external behavior of the system and should avoid, as far as possible, system design characteristics. The user requirements must be written using natural language, forms and simple intuitive diagrams.

System Requirements

System requirements are more detailed descriptions of their requirements. They may serve as the basis for the implementation of the system and should therefore be a complete and consentient specification of the whole system. They are used by software en-

gineers as the starting point for the system design requirement should state what the system should do and not how it should be implemented.

Structured language specifications

Structured language is a restricted form of natural language for writing system requirements. The advantage of this approach is that it maintains most of the expressiveness and understandability of natural language but ensures that some degree of uniformity is imposed on the specification. Structured language notations may limit the terminology used and may use templates to specify system requirements.

6.3. Requirements specification using a PDL

To counter the ambiguities in natural language specification, it is possible to describe requirements operationally using a program description language or PDL. A PDL is a language derived from programming language like java. It may contain additional, more abstract, constructs to increase its expressive power. The advantage of using a PDL is that it may be checked syntactically and semantically by software tools. Requirements omissions and inconsistencies may be inferred from the results of these checks.

Interface Specification

The vast majority of software systems must operate with other systems which have already been implemented and installed in an environment. If the new system and the existing sys-

tems must work together the interfaces of existing systems must be precisely specified. These specifications should be defined early in the process and included in the requirements document. There are three types of interface which have to be defined:-

1. Procedural interfaces where existing sub systems offer a range of services which are accessed by calling interface procedures.
2. Data structures which are passed from one sub-system to another.
3. Representations of data (such as the ordering of bits which have been established for an existing sub system.

6.4. The software requirements document

The software requirements document (sometimes called the software requirements specification or SRS) is the official statement of what is required of the system developers. It should include both the user requirements for a system and a detailed specification of the system requirements.

These are six requirements which a software requirements document should satisfy:-

1. It should specify only external system behavior
2. It should specify constraints on the implementation
3. It should be easy to change
4. It should serve as a reference tool for system maintainers
5. It should record forethought about the lifecycle of the system

tem

6. It should characterize acceptability responses to undesired events.

IEEE- standards suggests the following structure for requirements documents:

1. Introduction
   (a) purpose of the requirements documents
   (b) scope of the product
   (c) definitions, acronyms and abbreviations
   (d) references
   (e) Overview of the remainder of the document.
2. General description
   (a) Product perspective

(b) product functions
(c) user characteristics
(d) general constrains
(e) assumptions and dependencies

3. Specification Requirements - covering functional, non-functional and interface requirements
4. Appendices
5. Index

Revision Questions

Example ✎**.** Briefly describe the Software Requirement specification document (SRS)

Solution:

It act as an agreement between software vendor and the customer on the required features ,also, it helps in breaking down the requirements into estimable tasks and have good understanding of system requirements.It doesn't have to be a long document it depends on the application size. □

EXERCISE 1. ✍ The process of monitoring specific project results to determine if they comply with relevant quality standards is called

Quality Assurance

Quality Control

Quality Planning

Quality Review

References and Additional Reading Materials

1. Hans van Vliet (2005). Software Engineering: Principles and Practice (2nd ed.). John Wiley & Sons, Inc. ISBN: 0-471-97508-7
2. Ian Sommerville, Pete Sawyer(2005). Requirements Engineering: A Good Practice Guide. ISBN: 0-471-97444-7. John Wiley & Sons, Inc
3. Gerald Kotonya, Ian Sommerville (1998). Requirements Engineering: Processes and Techniques. John Wiley & Sons, Inc. ISBN: 0-471-97208-8